ST-ROBOT · TÀI LIỆU TỰ HỌC

Micro:bit

Ma trận LED, sự kiện nút bấm, cảm biến và sản phẩm tương tác.

6 bài học · Lý thuyết, ví dụ, thực hành, kiểm tra và tài liệu tham khảo

Lộ trình học tập

1. Vẽ ý tưởng trên 25 LED
2. Sự kiện và nút bấm
3. Cảm biến ánh sáng
4. Dự án: bộ đếm tương tác
5. Giao tiếp radio và giao thức bản tin
6. Dự án: bộ đếm có kiểm thử sự kiện

Cách sử dụng tài liệu

Đọc mục tiêu và kiến thức nền; tự dự đoán trước ví dụ; thực hành theo các bước; ghi lại dữ liệu và giải thích sai lệch. Sau khi tự làm, đối chiếu đáp án và tự kiểm tra. Phòng lab bên ngoài không tự đồng bộ điểm; hoạt động mô phỏng không thay thế kiểm tra an toàn phần cứng thật.

Bài 1/6 · Vẽ ý tưởng trên 25 LED

Chạy hình chữ thập trên 25 LED rồi tự đổi thành chữ L bằng tọa độ trong micro:bit Python Editor.

Chuẩn bị

Chưa cần học lập trình trước. Biết đếm 0,1,2,3,4 và nhận biết trái/phải, trên/dưới. Chuẩn bị một lưới giấy 5×5; chưa cần micro:bit thật.

Sau bài học, bạn có thể

1. Xác định đúng gốc tọa độ và phạm vi x,y = 0..4.

2. Chuyển một hình đơn giản thành danh sách điểm không trùng.

3. Xóa trạng thái cũ trước khi vẽ một hình mới.

Cùng hiểu bài từ đầu

Bạn có thể bắt đầu bài này bằng giấy kẻ ô, chưa cần biết vòng lặp. Hãy xem màn hình micro:bit là 25 vị trí cố định: chương trình chọn vị trí nào thì thay độ sáng của LED ở đó. Khi biến hình vẽ thành danh sách tọa độ, bạn đang biến một ý tưởng nhìn thấy được thành dữ liệu mà máy xử lý được. Việc đầu tiên là chỉ đúng một điểm, sau đó mới ghép nhiều điểm thành hình.

Hãy tưởng tượng 25 ngăn tủ được xếp thành 5 hàng và 5 cột. Muốn tìm một ngăn, bạn chỉ cột trước rồi hàng sau.

Cặp (x,y) chính là địa chỉ của LED: x đi ngang từ trái, y đi xuống từ trên. Cả hai bắt đầu ở 0; điểm (0,0) là góc trên trái và (4,4) là góc dưới phải.

Giới hạn của ví von: Các ngăn tủ chỉ giúp hiểu địa chỉ. LED còn có độ sáng; giá trị 9 là mức trong API, không phải 9 volt hay 9 lux. Các tọa độ này cũng không phải vị trí ngoài đời của robot.

Những từ cần hiểu

1. Tọa độ (x,y): Hai số xác định một điểm. Trong ma trận này x là cột và y là hàng, đều thuộc 0..4; đổi thứ tự hai số thường đổi vị trí.

2. Điểm ảnh: Một vị trí phát sáng trên màn hình; ở đây một điểm ảnh tương ứng một LED. Nhiều điểm tạo thành hình.

3. Độ sáng b: Số nguyên từ 0 đến 9 trong display.set_pixel: 0 tắt, 9 sáng nhất theo API. Nó không cho biết chính xác lượng ánh sáng phát ra.

4. Xóa màn hình: display.clear() đưa mọi điểm về tắt. Nếu chỉ vẽ thêm mà không xóa, những LED đã bật trước đó vẫn có thể còn sáng.

Từ hình chữ L đến danh sách điểm

Bắt đầu với: Màn hình ban đầu trống; cần sáng toàn bộ cột trái và hàng dưới ở mức 9.

Vẽ cột trái

Cột trái có x = 0. Đi lần lượt qua y = 0,1,2,3,4: được (0,0), (0,1), (0,2), (0,3), (0,4). Có 5 vị trí.

Nối hàng dưới

Hàng dưới có y = 4. Thêm (1,4), (2,4), (3,4), (4,4). Không cần thêm lại (0,4) vì đó đã là điểm cuối của cột trái.

Cho máy vẽ

Sau khi xóa màn hình, danh sách 9 điểm được đọc lần lượt. Mỗi lần gọi đặt một điểm; những điểm trước không tự bị xóa.

Đối chiếu

Đếm LED theo hình: cột 5 điểm và phần hàng nhô ra 4 điểm. Nếu hình nằm ở góc khác, kiểm tra hướng tăng của x,y trước khi sửa vòng lặp.

Kết quả: Có 9 LED sáng tạo chữ L. Chín là số vị trí khác nhau, không phải số lần có thể gọi hàm vẽ.

Hai nhóm giao nhau tại (0,4), nên số điểm là 5 + 5 − 1 = 9. Máy lưu trạng thái của từng điểm; vẽ cùng điểm hai lần không tạo thêm một LED.

Tự giải thích rồi đối chiếu

Điểm (4,0) nằm ở đâu? Vì sao không dùng (5,0)?

Đó là góc trên phải. Có 5 cột nhưng chỉ số là 0,1,2,3,4; 5 nằm ngoài phạm vi.

Giảm b từ 9 xuống 3 có làm hình chữ L ít điểm hơn không?

Không, nếu vẫn vẽ đủ 9 tọa độ thì hình có cùng các điểm, nhưng các điểm được đặt ở mức sáng thấp hơn.

Chọn độ sâu phù hợp

Mới bắt đầu · Bắt đầu bằng một LED

Vẽ lưới rồi chỉ (0,0), (4,0), (0,4). Sau khi quen góc và hướng, hãy nối thành chữ L; chưa cần tự viết vòng for.

Dự đoán vị trí (1,3), vẽ đúng một điểm rồi đối chiếu.

Đã học cơ bản · Hiểu danh sách và vòng lặp

Danh sách giữ dữ liệu hình; vòng for dùng lại một thao tác cho từng cặp tọa độ. Thay dữ liệu có thể đổi hình mà không đổi cách vẽ.

Tạo dấu cộng năm điểm và giải thích vì sao điểm giữa chỉ xuất hiện một lần trong danh sách.

Muốn tìm hiểu sâu · Xem hình như một tập điểm

Hình có thể biểu diễn bằng tập các cặp tọa độ. Khi ghép hình A và B, điểm chung phải được trừ khỏi tổng nếu đếm số LED; mức sáng của điểm vẽ lại theo lần đặt sau.

Tạo phép đối xứng trái–phải bằng (x,y) → (4−x,y), rồi dự đoán chữ L đổi như thế nào.

Kiến thức trọng tâm

Việc đầu tiên: vẽ bằng một danh sách điểm

micro:bit có 25 đèn LED xếp thành 5 hàng và 5 cột. Một cặp (x,y) chọn một đèn: x là cột từ trái sang phải, y là hàng từ trên xuống. Gốc (0,0) ở góc trên trái; mỗi chỉ số nhận 0,1,2,3,4. Hãy vẽ hình trên giấy trước rồi đối chiếu hình mô phỏng.

Lệnh MicroPython thay đổi màn hình

display.clear() tắt toàn bộ LED. display.set_pixel(x,y,b) đặt độ sáng b của một điểm từ 0 (tắt) đến 9 (sáng nhất). Bật thêm một điểm không tự xóa điểm cũ. Danh sách points trong mẫu chứa các cặp tọa độ; vòng for x, y in points lần lượt lấy mỗi cặp để vẽ. Ở lượt đầu chỉ cần đổi danh sách điểm, giữ nguyên khung chương trình.

Đọc thêm · Một hình, nhiều cách viết

Có thể tạo hình bằng nhiều lệnh display.set_pixel hoặc bằng Image của MicroPython. MakeCode JavaScript là lựa chọn khác: led.plot và basic.clearScreen thuộc MakeCode, không dùng trực tiếp trong micro:bit Python Editor. Python của MakeCode và MicroPython cũng là hai môi trường khác nhau.

Ví dụ có lời giải

Chữ thập 5 điểm gồm (2,2), (1,2), (3,2), (2,1), (2,3). Điểm giữa chỉ tính một lần. Chữ L gồm cột x=0 và hàng y=4: 5 + 5 − 1 = 9 LED vì (0,4) nằm trong cả hai nhóm.

Lỗi thường gặp và cách tránh

1. Tọa độ (5,0) không phải góc trên phải; góc đó là (4,0).

2. Vẽ lại một điểm đang sáng không tăng số LED sáng; chuyển editor không có nghĩa các API tự tương thích.

Các bước thực hành

1. Dự đoán: vẽ lưới 5×5, ghi x từ trái sang phải và y từ trên xuống, đều từ 0 đến 4. Đánh dấu năm điểm (2,2),(1,2),(3,2),(2,1),(2,3) rồi đoán hình sẽ hiện.

2. Thao tác: vào Lab → micro:bit Python Editor, dùng mã MicroPython của bài và chọn Run. Giữ nguyên mã ở lượt đầu để thấy một kết quả hoàn chỉnh.

3. Quan sát: so hình trên micro:bit mô phỏng với giấy; đếm năm LED sáng và chỉ điểm giữa (2,2). Nếu khác, kiểm tra lại thứ tự x,y.

4. Thay một việc: thay danh sách points bằng các điểm của chữ L ở bài tập. Giữ display.clear(), Run lại và đối chiếu số LED; chưa cần tự viết vòng for.

5. Giải thích: viết một câu về tác dụng của display.clear() và một câu vì sao chỉ số cuối là 4. Lưu mã hoặc ảnh cùng bảng dự đoán–quan sát.

6. Mở rộng: thử độ sáng 3 thay cho 9 trong display.set_pixel. Biến thể MakeCode JavaScript chỉ dùng trong MakeCode, có API riêng.

Mã mẫu · MicroPython · micro:bit Python Editor

```
from microbit import *

display.clear()
points = [(2, 2), (1, 2), (3, 2), (2, 1), (2, 3)]
for x, y in points:
  display.set_pixel(x, y, 9)  # brightness: 0..9, coordinates: 0..4
```

Thay thế · MakeCode JavaScript · MakeCode JavaScript

```
basic.clearScreen()
led.plot(2, 2)
led.plot(1, 2)
led.plot(3, 2)
led.plot(2, 1)
led.plot(2, 3)
```

Chọn và sử dụng phòng lab

Mở micro:bit Python Editor trong Lab, kiểm tra mã bắt đầu bằng from microbit import * rồi chọn Run. Quan sát chữ thập 5 điểm; thay danh sách points để vẽ chữ L rồi Run lại. display.set_pixel(x,y,9) bật một điểm với tọa độ 0..4; 9 là độ sáng lớn nhất trên thang 0..9. Bàn ma trận trong trang là mô hình theo thao tác, không chạy mã Python. Biến thể MakeCode JavaScript chỉ chạy trong MakeCode.

Bài tập áp dụng

Vẽ chữ L gồm cột trái và hàng dưới trên giấy; dự đoán số LED sáng. Thay danh sách points trong mã Python bằng các tọa độ đó, Run rồi ghi số đếm quan sát và giải thích điểm góc chỉ tính một lần. Phần thêm: giải thích vì sao (5,0) nằm ngoài ma trận.

Gợi ý đối chiếu sau khi tự làm

points = [(0,0),(0,1),(0,2),(0,3),(0,4),(1,4),(2,4),(3,4),(4,4)]. Có 9 LED: hai đoạn 5 điểm dùng chung góc (0,4), nên 5 + 5 − 1 = 9. Chỉ số hợp lệ là 0..4 nên x=5 ở ngoài ma trận. Giữ display.clear() trước khi vẽ để xóa hình cũ.

Tự kiểm tra mức hiểu

1. Điểm giữa (2,2) nằm ở cột và hàng thứ mấy khi đếm từ 1?

2. Vì sao hai đoạn 5 điểm tạo chữ L chỉ sáng 9 LED?

Câu hỏi trắc nghiệm

Tọa độ tâm ma trận 5 × 5 là...

1. (0,0)

2. (2,2)

3. (5,5)

Đáp án: 2. Với chỉ số 0,1,2,3,4 trên mỗi trục, chỉ số giữa là 2 nên tâm nằm tại (2,2).

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

MicroPython micro:bit — API chính thức

https://microbit-micropython.readthedocs.io/en/v2-docs/

MicroPython micro:bit — ma trận và read_light_level

https://microbit-micropython.readthedocs.io/en/v2-docs/display.html

micro:bit Python Editor — soạn thảo và mô phỏng

https://python.microbit.org/v/3/

Thay thế · MakeCode micro:bit — tọa độ LED

https://makecode.microbit.org/reference/led/plot

Thay thế · MakeCode micro:bit — xóa màn hình

https://makecode.microbit.org/reference/basic/clear-screen

Bài 2/6 · Sự kiện và nút bấm

Dùng vòng lặp MicroPython để A hiện trái tim, B xóa màn hình; kiểm tra kết quả theo thứ tự bấm.

Chuẩn bị

Đã chạy hình trong micro:bit Python Editor. Nhận biết display.clear(), nút A/B và ý nghĩa "nếu ... thì ..." của if.

Sau bài học, bạn có thể

1. Chỉ được phần chạy một lần ở đầu và phần lặp kiểm tra nút trong while True.

2. Kiểm tra A hiển thị, B xóa với nhiều thứ tự bấm.

3. Phân biệt dấu đã bấm từ was_pressed() với trạng thái đang giữ từ is_pressed().

Cùng hiểu bài từ đầu

Một nút bấm là đầu vào, còn hình trên LED là đầu ra. Giữa hai bên có chương trình liên tục kiểm tra xem vừa có điều gì xảy ra. Bài này giúp bạn tách ba chuyện dễ nhầm: nút đang bị giữ, đã có một lần bấm, và số lần đã bấm. Ta bắt đầu với quy tắc đơn giản: A hiện trái tim, B xóa. Sau đó theo dõi từng lần thay đổi thay vì chỉ nhìn màn hình cuối.

Một người trực bảng thông báo thường xuyên kiểm tra hộp thư: có phiếu A thì treo hình, có phiếu B thì gỡ hình.

while True là những lượt kiểm tra lặp lại. was_pressed() giống một dấu báo "đã có ít nhất một phiếu" từ lần kiểm tra trước; đọc dấu thì dấu đó được xóa.

Giới hạn của ví von: Đây không phải hàng đợi giữ riêng từng lần bấm. was_pressed() không cho biết có một hay ba lần bấm; cần get_presses() nếu muốn lấy số lần.

Những từ cần hiểu

1. Đầu vào và đầu ra: Nút A/B cung cấp thông tin vào; chương trình chọn hành động; ma trận LED thể hiện kết quả. Nút không tự quyết định biểu tượng nếu chưa có quy tắc.

2. Vòng lặp: Nhóm lệnh được thực hiện nhiều lượt. while True tiếp tục kiểm tra nút cho đến khi chương trình bị dừng hoặc khởi động lại.

3. was_pressed(): Trả lời có ít nhất một lần bấm kể từ lần đọc trước hay không, rồi xóa dấu đã bấm. Kết quả là True hoặc False, không phải số lần.

4. is_pressed(): Cho biết nút có đang được giữ tại thời điểm đọc. Giữ lâu có thể cho True qua nhiều lượt; đó không đồng nghĩa nhiều lần bấm mới.

Theo dõi chuỗi A → B → A

Bắt đầu với: Màn hình trống khi khởi động; bấm từng nút rời nhau, chờ nhìn thấy phản hồi trước lần tiếp theo.

Sau A thứ nhất

Lượt đọc gặp dấu A, nhánh A đặt Image.HEART lên màn hình. Trạng thái mới là trái tim.

Sau B

Lượt sau gặp dấu B, nhánh B gọi display.clear(). Trạng thái mới là trống; nó không "trừ một trái tim", mà đặt mọi LED về tắt.

Sau A cuối

Một lần bấm A mới tạo dấu mới. Chương trình lại hiện cùng hình trái tim; không cần nhớ đã hiện hình này bao nhiêu lần.

Kết quả: Các trạng thái lần lượt là trống → trái tim → trống → trái tim.

Mỗi thao tác ghi trạng thái mới lên cùng một màn hình. Nếu A và B đều được ghi nhận trong cùng lượt, mẫu xử lý A trước rồi B, nên cuối lượt là trống; đây là quy tắc riêng của thứ tự lệnh trong bài.

Tự giải thích rồi đối chiếu

Bấm A hai lần liên tiếp có tạo hai hình trái tim cạnh nhau không?

Không. Cả hai lần cùng đặt một biểu tượng lên cùng ma trận nên trạng thái vẫn là một trái tim.

Vì sao giữ A không có nghĩa chương trình nhận một lần bấm mới ở mọi lượt?

was_pressed() đọc dấu về lần bấm mới, khác với is_pressed() kiểm tra trạng thái đang giữ.

Chọn độ sâu phù hợp

Mới bắt đầu · Theo dấu từng thao tác

Viết "trống" hoặc "tim" sau mỗi nút trên giấy. Quan sát chậm giúp nối một đầu vào với một đầu ra trước khi nghĩ đến nhiều nút gần nhau.

Dự đoán B,B,A rồi thử và ghi đủ ba trạng thái.

Đã học cơ bản · Đọc luồng chương trình

Phần trước while True chạy một lần; phần thụt vào chạy nhiều lần. sleep(20) nghỉ 20 ms giữa các lượt, không tự xóa màn hình.

Giải thích vì sao chuyển display.clear() vào đầu mỗi lượt sẽ làm hành vi hiển thị khác.

Muốn tìm hiểu sâu · Xét các sự kiện cùng lượt

Hai if độc lập cho phép cả hai nhánh chạy trong một lượt. Chính sách ưu tiên phải được suy ra từ thứ tự lệnh, không chỉ từ tốc độ bấm bằng tay.

Lập bảng bốn trường hợp dấu A/B: không/không, có/không, không/có, có/có; ghi màn hình cuối mỗi lượt.

Kiến thức trọng tâm

Một lần khởi động, nhiều lượt kiểm tra

Trong MicroPython, display.clear() ở đầu chạy một lần khi chương trình bắt đầu. while True lặp các lệnh thụt vào bên dưới. Mỗi lượt, if button_a.was_pressed() hỏi có lần bấm A nào từ lần đọc trước không; nếu có thì hiện trái tim. sleep(20) ở cuối vòng cho biết chương trình nghỉ ngắn trước lượt kiểm tra tiếp theo.

Thứ tự đầu vào quyết định màn hình

A/B là đầu vào, LED là đầu ra. Mẫu xử lý A trước rồi B: A hiện Image.HEART, B gọi display.clear(). Các lần bấm tuần tự cho trạng thái theo nút cuối. Nếu A và B đều có dấu đã bấm trong cùng lượt đọc thì B xóa sau A. sleep(20) cho chương trình nghỉ 20 ms trước lượt kiểm tra tiếp theo.

Đọc thêm · Dấu đã bấm và trạng thái đang giữ

was_pressed() trả True khi có ít nhất một lần bấm kể từ lần gọi trước và xóa dấu đó; nó không cho biết đã bấm bao nhiêu lần. is_pressed() cho biết nút đang được giữ tại thời điểm đọc. get_presses() trả số lần bấm từ lần gọi trước. Biến thể MakeCode dùng input.onButtonPressed trong MakeCode; không dán API đó vào micro:bit Python Editor.

Ví dụ có lời giải

Ban đầu xóa màn hình. Chuỗi A,A,B kết thúc ở màn hình trống; B,A kết thúc ở trái tim. Hai lần A không tạo "hai trái tim" vì cùng ghi một biểu tượng lên cùng ma trận. Đây là phép kiểm tra trạng thái chứ chưa phải một bộ đếm.

Lỗi thường gặp và cách tránh

1. while True không tự làm nhánh A hiển thị mỗi vòng: còn phải thỏa điều kiện was_pressed().

2. Hai lần bấm A trong chương trình này không phải bộ đếm; chưa có biến lưu số lần bấm.

Các bước thực hành

1. Dự đoán: ghi quy tắc A → trái tim, B → trống; dự đoán trạng thái cuối của A,A,B và B,A từ màn hình trống.

2. Thao tác: mở Lab → micro:bit Python Editor, chạy mã của bài và kiểm tra màn hình ban đầu trống.

3. Quan sát: bấm từng nút theo hai chuỗi, chờ thấy phản hồi rồi bấm tiếp. Ghi trạng thái sau mỗi lần bấm và đối chiếu dự đoán.

4. Thử tiếp: giữ A, thả ra rồi bấm rời thêm lần nữa. Đọc chú thích was_pressed(): chương trình đọc dấu đã bấm, không liên tục coi trạng thái đang giữ là một lần bấm mới.

5. Giải thích: chỉ display.clear() ở đầu chương trình, while True lặp việc kiểm tra và hai nhánh if thay đổi hình. Nếu cả hai nút cùng được ghi nhận trong một lượt đọc, A xử lý trước rồi B xóa nên B được ưu tiên.

Mã mẫu · MicroPython · micro:bit Python Editor

```
from microbit import *

display.clear()
while True:
  # Edge-latched presses, not a repeated test of the held state.
  if button_a.was_pressed():
    display.show(Image.HEART)
  if button_b.was_pressed():
    display.clear()
  # If both were latched in one poll, B wins in this example.
  sleep(20)
```

Thay thế · MakeCode JavaScript · MakeCode JavaScript

```
basic.clearScreen()
input.onButtonPressed(Button.A, function () {
  basic.showIcon(IconNames.Heart)
})
input.onButtonPressed(Button.B, function () {
  basic.clearScreen()
})
```

Chọn và sử dụng phòng lab

Trong Lab, chọn micro:bit Python Editor và chạy mã MicroPython. Bấm A/B trên bo mô phỏng: A hiện trái tim, B xóa. Thử A,A,B và B,A theo từng bước. Hai nhánh if được đọc trong while True, nghỉ 20 ms mỗi vòng; nếu cả hai nút cùng được ghi nhận trong một lượt đọc thì nhánh B chạy sau và màn hình trống. Mô hình nút bấm trong trang chỉ minh họa quy tắc phản hồi; thử nút với chương trình ở Python Editor.

Bài tập áp dụng

Từ màn hình trống, dự đoán rồi thử A,B,A và B,B. Đổi Image.HEART thành Image.HAPPY trong nhánh A, Run lại và kiểm tra A rồi B. Viết một câu về cách was_pressed() khác với kiểm tra đang giữ nút.

Gợi ý đối chiếu sau khi tự làm

A,B,A kết thúc ở trái tim; B,B vẫn trống. Sau khi đổi thành display.show(Image.HAPPY), A hiện mặt cười và B vẫn xóa. was_pressed() đọc rồi xóa dấu có lần bấm mới; is_pressed() đọc trạng thái nút đang được giữ. Chương trình này chưa lưu số lần bấm.

Tự kiểm tra mức hiểu

1. Vì sao phải thử cả B,A và A,B?

2. Muốn nhớ số lần A đã bấm thì cần bổ sung loại dữ liệu/trạng thái nào?

Câu hỏi trắc nghiệm

Với if button_a.was_pressed(), lệnh hiện trái tim chạy khi nào?

1. Chỉ khi tắt nguồn

2. Khi lần đọc ghi nhận A đã được bấm kể từ lần đọc trước

3. Mỗi giây mặc định

Đáp án: 2. was_pressed() trả True nếu A đã được bấm kể từ lần gọi trước (hoặc từ lúc khởi động), rồi xóa dấu đã bấm. Nhánh if hiện trái tim khi giá trị đó là True; nó không tự chạy mỗi giây.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

micro:bit — các nút trên thiết bị

https://microbit.org/get-started/features/buttons/

MicroPython micro:bit — API chính thức

https://microbit-micropython.readthedocs.io/en/v2-docs/

MicroPython micro:bit — nút bấm và get_presses

https://microbit-micropython.readthedocs.io/en/v2-docs/button.html

micro:bit Python Editor — soạn thảo và mô phỏng

https://python.microbit.org/v/3/

Thay thế · MakeCode micro:bit — sự kiện nút bấm

https://makecode.microbit.org/reference/input/on-button-pressed

Bài 3/6 · Cảm biến ánh sáng

Sử dụng mức sáng tương đối để thiết kế quy tắc ngưỡng và kiểm thử tại biên.

Chuẩn bị

Biết if/else, so sánh < và lệnh hiển thị; hoàn thành bài ma trận.

Sau bài học, bạn có thể

1. Phân biệt giá trị tương đối 0..255 của display.read_light_level() với lux.

2. Dự đoán hai nhánh ở dưới, bằng và trên ngưỡng.

3. Chọn ngưỡng từ nhiều số đo trong điều kiện sử dụng.

Cùng hiểu bài từ đầu

Cảm biến không trả về câu "phòng tối", mà trả về một con số. Chương trình dùng quy tắc do bạn chọn để biến số đó thành hành động. Trong bài, số nhỏ hơn 80 làm hiện trái tim; số bằng hoặc lớn hơn 80 làm màn hình trống. Muốn hiểu quy tắc, hãy thử ngay sát ranh giới trước, rồi mới khảo sát ngưỡng có phù hợp với nơi dùng hay không.

Một vạch trên thước giúp chia các số đo thành hai nhóm: phía dưới vạch và từ vạch trở lên.

Ngưỡng 80 đóng vai trò ranh giới trong điều kiện level < 80. Cảm biến cung cấp level; dấu so sánh quyết định nhánh.

Giới hạn của ví von: Thang 0..255 ở đây là mức sáng tương đối, không phải thước đo lux đã hiệu chuẩn. Cùng số đọc không bảo đảm cùng độ rọi trên mọi bo hoặc mọi cách đặt.

Những từ cần hiểu

1. Mức sáng tương đối: Số từ 0 đến 255 do display.read_light_level() trả về. Số thấp biểu thị tối hơn trên thang này; không gắn đơn vị lux cho nó.

2. Ngưỡng: Giá trị dùng để chia đầu vào thành các nhánh. Ngưỡng phù hợp phải dựa vào số đo ở điều kiện dự định dùng.

3. Giá trị biên: Số nằm đúng ranh giới hoặc sát ranh giới, như 79,80,81. Chúng giúp phát hiện nhầm < với ≤.

4. Khởi tạo phép đo: Phép đọc đầu tiên có thể là 0 khi chưa sẵn sàng. Mẫu đọc khởi tạo rồi chờ 200 ms; không nên diễn giải riêng số đầu đó là phòng tối.

Thử ba số để hiểu một dấu so sánh

Bắt đầu với: Điều kiện level < 80; sau khi phép đo đã sẵn sàng, lần lượt đưa đầu vào 79, 80, 81.

Đầu vào 79

79 nhỏ hơn 80 nên điều kiện đúng. Chương trình hiện trái tim.

Đầu vào 80

80 bằng 80 nhưng không nhỏ hơn. Điều kiện sai, nên chạy nhánh xóa màn hình.

Đầu vào 81

81 lớn hơn 80 nên cũng chạy nhánh xóa. Ghi lại cả ba kết quả để tách kiểm tra quy tắc khỏi việc chọn ngưỡng thực tế.

Kết quả: Bảng kết quả: 79 → tim; 80 → trống; 81 → trống. Các số là mức trên thang tương đối, không có đơn vị lux.

Chương trình chỉ xét một điều kiện logic. Simulator cho phép đặt số để kiểm tra điều kiện; việc đó chưa chứng minh 80 phân biệt tốt bàn học sáng và góc tối ngoài đời.

Tự giải thích rồi đối chiếu

Đổi điều kiện thành level ≤ 80 thì kết quả nào trong bộ ba đổi?

Chỉ mức 80 đổi từ trống sang trái tim; 79 vẫn hiện tim, 81 vẫn trống.

Hai nhóm số đo thực chồng lấn nhiều thì đặt một ngưỡng có chắc phân biệt ổn định không?

Không. Cần xem cách đặt, điều kiện sáng và độ dao động; có thể phải thay phương pháp hoặc bổ sung dữ liệu.

Chọn độ sâu phù hợp

Mới bắt đầu · Bắt đầu bằng bảng ba dòng

Bạn chưa cần đo phòng thật. Viết đầu vào, kết quả so sánh và hình kỳ vọng cho từng số quanh ngưỡng, rồi dùng simulator đối chiếu.

Với ngưỡng 50 và dấu <, thử 49,50,51 rồi giải thích riêng trường hợp 50.

Đã học cơ bản · Chọn ngưỡng từ dữ liệu

Đo nhiều lần ở hai điều kiện muốn phân biệt. Với nhóm giả định 20..35 và 120..160, có nhiều ngưỡng nằm giữa; phải thử lại khi điều kiện đổi.

Đề xuất ngưỡng 80, rồi nêu hai tình huống cần kiểm tra trước khi dùng thực.

Muốn tìm hiểu sâu · Xử lý dao động quanh biên

Nếu số đọc dao động 79,80,79,81, một ngưỡng có thể làm đầu ra đổi liên tục. Hai ngưỡng với việc nhớ trạng thái có thể giảm đổi qua lại, nhưng cần đặc tả rõ điều kiện bật/tắt.

Thiết kế bật khi level < 70, tắt khi level > 90, giữ trạng thái ở giữa; chạy tay chuỗi 65,75,85,95,80.

Kiến thức trọng tâm

Đang đo đại lượng nào?

display.read_light_level() trong MicroPython trả mức sáng tương đối 0..255: số nhỏ hơn biểu thị tối hơn trên thang này. Nó không phải lux và không đủ để so sánh độ rọi tuyệt đối giữa các cảm biến. micro:bit dùng ma trận LED để cảm nhận ánh sáng; phép đọc đầu tiên có thể là 0 khi phép đo chưa sẵn sàng. Mẫu đọc khởi tạo rồi nghỉ 200 ms trước khi dùng kết quả.

Quy tắc ngưỡng có biên rõ ràng

Biến level nhận kết quả từ display.read_light_level(). Với if level < 80, giá trị 79 hiện trái tim, 80 và 81 vào else để xóa màn hình. while True đọc lại định kỳ, sleep(100) nghỉ 100 ms mỗi vòng. Viết bảng ba giá trị này trước khi chạy giúp phát hiện sai dấu.

Đọc thêm · Khảo sát rồi chọn ngưỡng

Thử ít nhất hai điều kiện sử dụng như bàn học sáng và vị trí che bớt ánh sáng. Nếu hai nhóm số đo chồng lấn nhiều, một ngưỡng đơn giản khó phân biệt ổn định. Có thể cần đổi vị trí đo hoặc dùng hai ngưỡng ở bài nâng cao. Trên simulator, thanh điều chỉnh chỉ mô phỏng đầu vào, không đo ánh sáng của căn phòng thật.

Ví dụ có lời giải

Với if level < 80: mức 79 → hiện trái tim; mức 80 → xóa; mức 81 → xóa. Dữ liệu giả định ở vị trí tối là 25,30,35 và vị trí sáng là 140,150,160 có thể tách bằng 80 trong khảo sát đó; không suy ra 80 đúng với mọi thiết bị và phòng.

Lỗi thường gặp và cách tránh

1. Giá trị 100 không có nghĩa chắc chắn là 100 lux.

2. Mức sáng simulator không phải kết quả đo ánh sáng qua camera của máy tính.

Các bước thực hành

1. Dự đoán: với điều kiện level < 80, lập bảng 79,80,81; ghi mỗi giá trị sẽ hiện trái tim hay màn hình trống.

2. Thao tác: chạy mã MicroPython trong Lab → Python Editor. Sau lượt khởi tạo cảm biến, thay mức ánh sáng simulator và thử lần lượt 79,80,81.

3. Quan sát: ghi hình hiện ra ở mỗi mức và đối chiếu bảng. Ở mức bằng 80, kiểm tra đặc biệt dấu < không bao gồm bằng.

4. Thử tiếp: đổi 80 thành 50 trong if level < 80, Run lại rồi thử 49,50,51.

5. Giải thích: nêu vì sao số đọc là mức tương đối, không phải lux. Nếu có bo thật, chờ cảm biến khởi tạo rồi ghi nhiều lần đọc ở ít nhất hai vị trí trước khi đề xuất ngưỡng.

Mã mẫu · MicroPython · micro:bit Python Editor

```
from microbit import *

# Prime the first light reading; the matrix also senses light.
display.read_light_level()
sleep(200)
while True:
  level = display.read_light_level()  # relative value 0..255, not lux
  if level < 80:
    display.show(Image.HEART)
  else:
    display.clear()
  sleep(100)
```

Thay thế · MakeCode JavaScript · MakeCode JavaScript

```
input.lightLevel() // Khoi dong phep doc truoc vong lap
basic.pause(200)
basic.forever(function () {
  let level = input.lightLevel()
  if (level < 80) {
    basic.showIcon(IconNames.Heart, 0)
  } else {
    basic.clearScreen()
  }
  basic.pause(100)
})
```

Chọn và sử dụng phòng lab

Trong Lab → micro:bit Python Editor, chạy mã có display.read_light_level(). Thay ánh sáng của simulator, thử 79,80,81 với ngưỡng 80 rồi đối chiếu hình. Chương trình đọc một lần và nghỉ 200 ms để khởi tạo trước vòng lặp; sau đó cập nhật mỗi khoảng 100 ms. Bàn mô hình ngưỡng trong trang chỉ tính nhánh từ giá trị nhập, không chạy Python hay đo ánh sáng căn phòng.

Bài tập áp dụng

Đổi điều kiện thành level < 50. Dự đoán rồi thử 49,50,51 trong Python Editor, ghi hình quan sát. Từ hai nhóm đo giả định 20..35 và 120..160, đề xuất một ngưỡng và nêu điều kiện còn cần thử.

Gợi ý đối chiếu sau khi tự làm

49 thỏa nhánh tối; 50 và 51 không thỏa. Có thể thử 80 vì nằm giữa hai nhóm trong dữ liệu đã cho; cần kiểm tra thêm ở vị trí, thời gian, bo và điều kiện sử dụng mới, không coi đó là một giá trị lux.

Tự kiểm tra mức hiểu

1. Nếu đổi điều kiện thành <= 80 thì ca nào trong 79,80,81 đổi nhánh?

2. Nếu nhóm sáng và tối cùng thường trả 70..90, vì sao một ngưỡng không đủ tin cậy?

Câu hỏi trắc nghiệm

Giá trị display.read_light_level() từ 0 đến 255 được hiểu là...

1. Lux đã hiệu chuẩn

2. Mức sáng tương đối

3. Khoảng cách cm

Đáp án: 2. API trả một mức sáng tương đối trên thang 0..255. Thang số này không được gắn trực tiếp với đơn vị lux đã hiệu chuẩn.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

micro:bit — cảm nhận ánh sáng

https://microbit.org/get-started/features/light-sensor/

MicroPython micro:bit — API chính thức

https://microbit-micropython.readthedocs.io/en/v2-docs/

MicroPython micro:bit — ma trận và read_light_level

https://microbit-micropython.readthedocs.io/en/v2-docs/display.html

micro:bit Python Editor — soạn thảo và mô phỏng

https://python.microbit.org/v/3/

Thay thế · MakeCode micro:bit — mức ánh sáng

https://makecode.microbit.org/reference/input/light-level

Bài 4/6 · Dự án: bộ đếm tương tác

Dùng biến lưu số lần bấm và kiểm tra quy tắc tăng/đặt lại bằng bảng trạng thái.

Chuẩn bị

Đã chạy bài nút A/B trong micro:bit Python Editor. Hiểu if và while True; count là một tên biến để nhớ một số.

Sau bài học, bạn có thể

1. Khởi tạo count một lần trước vòng lặp và cập nhật từ số lần bấm đọc được.

2. Phân biệt giá trị count với hình số đang cuộn trên LED.

3. Kiểm tra reset khi đã đếm, khi đang 0 và khi A/B cùng được ghi nhận.

Cùng hiểu bài từ đầu

Bộ đếm khác bài hiện biểu tượng ở chỗ nó phải nhớ lịch sử. Mỗi lần nhận một số lần bấm A mới, chương trình cộng số đó vào count đang có. B đưa count về 0. Ta sẽ tách giá trị được nhớ trong biến khỏi chữ đang cuộn trên LED: hình có thể đang giữa lượt cuộn, nhưng count vẫn là một số xác định trong chương trình.

Hãy coi count như số ghi trên một tờ giấy: nhận thêm hai vé thì cộng hai; yêu cầu đặt lại thì xóa số cũ và ghi 0.

Biến giữ trạng thái giữa các lượt của vòng lặp. get_presses() lấy số lần A kể từ lần đọc trước; phép cộng dùng số đó cùng count cũ.

Giới hạn của ví von: Tờ giấy có thể còn sau khi cất đi, còn biến trong mẫu thì mất khi chạy lại hoặc mất nguồn. Muốn lưu bền vững cần một thiết kế khác.

Những từ cần hiểu

1. Biến count: Tên gắn với số đang được nhớ. count = 0 trước vòng lặp đặt trạng thái ban đầu một lần; các lượt sau dùng trạng thái đã cập nhật.

2. get_presses(): Trả số lần bấm kể từ lần gọi trước rồi đưa bộ đếm sự kiện của nút về 0. Không cộng cùng một lượt bấm mãi mãi.

3. Đặt lại: Gán count = 0 bất kể trước đó là bao nhiêu. Đặt lại hai lần vẫn là 0.

4. Hiển thị không chờ: wait=False cho phép vòng lặp tiếp tục khi số đang cuộn. Một ảnh LED giữa chừng không nhất thiết cho thấy toàn bộ số.

Cộng theo lượt đọc, không chỉ theo ảnh LED

Bắt đầu với: Ban đầu count = 0. Bấm rời nhau theo A,A,B,A; mỗi lần chờ đọc được số.

Hai lần A đầu

Lượt đầu lấy presses = 1: 0 + 1 = 1. Lượt sau lấy thêm 1: 1 + 1 = 2. Biến giữ 1 giữa hai lượt nên mới cộng dồn được.

Lần B

Có reset nên count được gán 0. Mẫu ưu tiên nhánh reset; nếu A cũng được ghi nhận trong cùng lượt thì không cộng các lần A của lượt đó.

Lần A cuối

Không có reset, presses = 1, nên count mới là 0 + 1 = 1. Yêu cầu hiển thị dùng giá trị 1 này.

Kết quả: Dãy giá trị count là 0 → 1 → 2 → 0 → 1. Chạy lại toàn bộ chương trình sẽ trở về 0.

Bộ đếm hoạt động nhờ ba phần: nhận sự kiện mới, dùng trạng thái cũ để tính trạng thái mới, rồi hiển thị. Khởi tạo count trong mỗi nhánh A sẽ làm mất lịch sử trước khi cộng.

Tự giải thích rồi đối chiếu

Một lượt đọc nhận presses = 3, count cũ là 5 và không reset. Count mới bằng bao nhiêu?

Bằng 8. get_presses() có thể trả nhiều lần bấm tích lũy giữa hai lượt đọc, không chỉ 0 hoặc 1.

Vì sao A,A không thể kiểm thử đúng nếu chỉ nhìn số cuối của A,A,B?

B cuối đặt lại 0 nên có thể che lỗi cộng dồn trước đó. Cần ghi cả trạng thái 1 rồi 2 trước B.

Chọn độ sâu phù hợp

Mới bắt đầu · Làm bảng trước và sau

Vẽ hai cột count trước/count sau. Mỗi lần A tăng một ở bộ thử chậm; mỗi lần B đưa về 0. Chưa cần bấm nhanh.

Chạy tay B,B,A,A từ 0 rồi đối chiếu từng bước với simulator.

Đã học cơ bản · Phân biệt số đếm và màn hình

display.scroll trình bày số bằng nhiều khung hình. Kiểm tra phải có mốc quan sát rõ; khi cần chẩn đoán có thể dùng đầu ra số thử nghiệm trong bản sao.

Giải thích trạng thái count và chữ đang cuộn có thể khác nhau về thời điểm cập nhật như thế nào.

Muốn tìm hiểu sâu · Đặc tả xung đột sự kiện

Một vòng có thể nhận cả presses > 0 và reset = True. Mẫu chọn reset, nên số lần A đã đọc trong lượt đó bị bỏ qua theo chính sách đã nêu.

Lập bảng với count cũ 7 cho bốn cặp (presses,reset): (0,False),(2,False),(0,True),(2,True).

Kiến thức trọng tâm

Biến là trạng thái được nhớ

count = 0 trước while True tạo trạng thái ban đầu một lần. Mỗi lượt, button_a.get_presses() trả số lần A từ lần gọi trước rồi đưa bộ đếm nút đó về 0; presses giữ số vừa lấy. Nếu không reset, count += presses cộng số này vào trạng thái cũ. Đặt count = 0 trong mọi nhánh A sẽ xóa lịch sử. Biến không tự lưu sau khi khởi động lại.

Đặt lại và hiển thị là hai việc riêng

button_b.was_pressed() cho biết có lần B kể từ lần đọc trước. Mẫu xét reset trước nhánh cộng A, nên B được ưu tiên khi cả hai được ghi nhận trong một lượt đọc. display.scroll(str(count), wait=False, loop=False) cuộn số mà vòng kiểm tra vẫn tiếp tục. Một ảnh chụp giữa lúc cuộn chưa cho biết toàn bộ count; thử chậm và đợi đọc rõ số ở lượt đầu.

Đọc thêm · Bảng chuyển trạng thái

Với các lần bấm rời nhau, từ n, A cho n+1; từ bất kỳ n, B cho 0. Hai lần B vẫn cho 0. Đây là số sự kiện nút đã nhận theo quy tắc của chương trình, chưa phải thiết bị đếm người hay sản phẩm thực tế. Mẫu không tự lưu bền vững. Biến thể MakeCode có API và cách xử lý sự kiện riêng; dùng nó trong MakeCode.

Ví dụ có lời giải

Ban đầu 0. Chuỗi A,A,B,A tạo 1,2,0,1. Chuỗi B,B,A tạo 0,0,1. Hai lần đặt lại liên tiếp không làm số âm và không cần trừ đi giá trị hiện tại.

Lỗi thường gặp và cách tránh

1. Đặt count = 0 ở đầu mỗi nhánh cộng A sẽ xóa lịch sử đếm.

2. Hiển thị một số không tự lưu số đó sau khi tắt hay khởi động lại.

Các bước thực hành

1. Dự đoán: lập bảng thao tác, count trước và count sau cho khởi động → A → A → B → B → A; ghi đầy đủ trước khi chạy.

2. Thao tác: chạy mã MicroPython trong Lab → Python Editor, xác nhận khởi động hiển thị 0.

3. Quan sát: bấm A hai lần, mỗi lần đợi thấy số rồi bấm tiếp; đối chiếu count = 1 rồi 2.

4. Thử B hai lần rồi A; ghi kết quả từng bước. Sau đó khởi động lại chương trình và kiểm tra về 0.

5. Giải thích: chỉ dòng count = 0 trước while True, get_presses() lấy số lần A đã bấm và nhánh reset đặt lại. Nếu A/B cùng được ghi nhận trong một lượt đọc, B được ưu tiên; không dùng thứ tự của các ca chậm để suy kết quả mọi ca bấm nhanh.

Mã mẫu · MicroPython · micro:bit Python Editor

```
from microbit import *

count = 0
display.show(str(count))
while True:
  # get_presses consumes the number of A presses since the last read.
  presses = button_a.get_presses()
  reset = button_b.was_pressed()
  if reset:
    count = 0
  elif presses:
    count += presses
  if reset or presses:
    # Non-blocking display keeps the polling loop responsive.
    display.scroll(str(count), wait=False, loop=False)
  # B has priority if A and B arrived within the same polling interval.
  sleep(20)
```

Thay thế · MakeCode JavaScript · MakeCode JavaScript

```
let count = 0
basic.showNumber(count)
input.onButtonPressed(Button.A, function () {
  count += 1
  basic.showNumber(count)
})
input.onButtonPressed(Button.B, function () {
  count = 0
  basic.showNumber(count)
})
```

Chọn và sử dụng phòng lab

Chạy mã MicroPython trong Lab → Python Editor. Thử A,A,B,B,A từng bước rồi Run lại để kiểm tra khởi tạo 0. get_presses() lấy và xóa số lần A tích lũy từ lần đọc trước; was_pressed() đọc dấu B. Nếu cả hai được ghi nhận trong cùng lượt đọc, B đặt count về 0 và các lần A của lượt đó không được cộng. display.scroll(..., wait=False) hiển thị không chặn; bấm chậm để đọc đủ số ở lần kiểm tra đầu. Bàn bộ đếm trong trang là mô hình theo thao tác, không chạy đoạn Python.

Bài tập áp dụng

Dự đoán rồi thử khởi động → A → A → B → B → A, ghi trạng thái mỗi bước. Trong bản sao, thêm count = 0 ngay trước count += presses ở nhánh A, thử A,A và giải thích lỗi trước khi khôi phục.

Gợi ý đối chiếu sau khi tự làm

Các trạng thái lần lượt 0,1,2,0,0,1. Trong bản sửa lỗi có chủ đích, nếu mỗi nhánh A lại gán count = 0 trước khi cộng thì hai lần A rời nhau cho 1,1 thay vì 1,2. Khởi tạo một lần trước vòng lặp để giữ lịch sử; xóa dòng thêm rồi thử lại.

Tự kiểm tra mức hiểu

1. Thao tác B,B khác gì phép trừ 1 hai lần?

2. Nếu simulator khởi động lại, ví dụ này có biết số trước đó không?

Câu hỏi trắc nghiệm

Sau A, A, B, A từ số 0, bộ đếm là...

1. 0

2. 1

3. 3

Đáp án: 2. A tăng từ 0 lên 1 rồi 2; B đặt lại 0; A cuối tăng lên 1. Phải xét thứ tự thao tác, không chỉ đếm tổng số chữ A.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

MicroPython micro:bit — API chính thức

https://microbit-micropython.readthedocs.io/en/v2-docs/

MicroPython micro:bit — nút bấm và get_presses

https://microbit-micropython.readthedocs.io/en/v2-docs/button.html

micro:bit Python Editor — soạn thảo và mô phỏng

https://python.microbit.org/v/3/

Thay thế · MakeCode micro:bit — sự kiện nút bấm

https://makecode.microbit.org/reference/input/on-button-pressed

Thay thế · MakeCode — biến số

https://makecode.microbit.org/blocks/variables/var

Bài 5/6 · Giao tiếp radio và giao thức bản tin

Thiết kế giao thức radio đơn giản có kiểm tra tên, miền dữ liệu và giới hạn bảo mật.

Chuẩn bị

Đã biết biến, if và vòng lặp MicroPython. Bắt đầu bằng bảng Radio của Python Editor để gửi chuỗi thử; cần hai micro:bit thật khi muốn kiểm tra truyền giữa hai bo.

Sau bài học, bạn có thể

1. Đặt cùng nhóm radio trên bên gửi và bên nhận.

2. Chỉ nhận đúng tên bản tin và miền số đã quy ước.

3. Phân biệt lọc nhóm, mã hóa và tình trạng mất dữ liệu.

Cùng hiểu bài từ đầu

Hai thiết bị chỉ hiểu nhau khi thống nhất cách viết bản tin. Chuỗi temp:23 không tự có ý nghĩa với radio: ta quy ước temp là tên trường và 23 là nhiệt độ giả định theo °C. Bên nhận phải kiểm tra trước khi dùng số. Bài này dạy cách giữ thông tin đúng gần nhất khi gặp bản tin lỗi, đồng thời nhận ra thông tin cũ không phải một phép đo mới.

Hình dung hai người trao phiếu theo mẫu “tên:một số”. Người nhận kiểm tra đúng mẫu và đúng tên trước khi chép số lên bảng.

Giao thức quy định cấu trúc temp:23, đơn vị °C, nhóm 7 và miền được chấp nhận −20..60. Nếu phiếu sai, số hợp lệ cũ trên bảng vẫn được giữ.

Giới hạn của ví von: Nhóm radio giống bộ lọc nhận, không phải phong bì khóa. Nó không mã hóa dữ liệu và không chứng minh ai đã gửi; bản tin có thể bị mất hoặc đến chậm.

Những từ cần hiểu

1. Giao thức: Các quy tắc hai bên cùng tuân theo: định dạng, tên trường, đơn vị, miền giá trị và cách xử lý lỗi. Cùng bật radio chưa đủ để hiểu dữ liệu.

2. None: radio.receive() trả None khi chưa có bản tin để đọc. Đây là trạng thái thiếu dữ liệu mới, khác số nhiệt độ 0 °C.

3. Kiểm tra miền: Chấp nhận số trong đoạn đóng [−20,60], có cả hai đầu mút. Miền này là yêu cầu của bài, không phải công bố độ chính xác của cảm biến.

4. last_valid: Biến giữ giá trị hợp lệ gần nhất. Giữ nguyên sau lỗi giúp tránh dùng dữ liệu sai, nhưng vẫn cần biết thời điểm nhận nếu muốn phát hiện quá hạn.

Giữ dữ liệu đúng qua các bản tin lỗi

Bắt đầu với: Bên nhận MicroPython dùng nhóm 7, last_valid đang là 21 °C. Các bản tin đến theo thứ tự temp:23, temp:999, other:22, temp:-20.

Nhận temp:23

Tách tại dấu : được tên temp và phần số 23. Đúng tên, đổi được thành số, nằm trong miền nên lưu 23 °C.

Nhận temp:999 và other:22

999 vượt miền nên bị loại. Bản tin tiếp có số 22 hợp miền nhưng tên other sai quy ước nên cũng bị loại. Giá trị lưu vẫn là 23 °C.

Nhận temp:-20

Tên đúng và −20 là biên được phép. Cập nhật last_valid thành −20 °C; không loại chỉ vì số có dấu âm.

Chưa có bản tin tiếp

Lần đọc trả None. Không ghi 0 vào last_valid; cũng không gọi −20 là số đo mới. Muốn biết đã cũ bao lâu cần thêm dấu thời gian.

Kết quả: Giá trị lưu lần lượt là 23, 23, 23, −20 °C. Chuỗi số cuộn là đầu ra tạm thời; dùng print thử nghiệm hoặc theo dõi biến để kiểm tra trạng thái sau bản tin bị loại.

Kiểm tra tách biệt giúp giải thích nguyên nhân loại dữ liệu: sai cấu trúc, sai tên, không đổi được sang số hoặc sai miền. Việc giữ số cũ bảo vệ trạng thái, nhưng không bảo đảm dữ liệu còn mới hay đúng về mặt vật lý.

Tự giải thích rồi đối chiếu

temp:60 và temp:61 khác nhau ra sao?

60 °C được nhận vì miền có cả biên 60; 61 °C bị loại và giữ last_valid cũ.

Hai bên cùng nhóm nhưng một bên dùng MakeCode sendValue, một bên nhận chuỗi MicroPython của bài có đủ để kết luận tương thích không?

Không. Cần cùng định dạng và cơ chế xử lý; bài yêu cầu một cặp MicroPython dùng chuỗi hoặc một cặp MakeCode riêng.

Chọn độ sâu phù hợp

Mới bắt đầu · Đọc bản tin như một phiếu

Gạch dưới tên và khoanh phần số. Kiểm tra từng điều kiện theo thứ tự trước khi đổi giá trị đang lưu; chưa cần hai bo thật để chạy tay quy tắc.

Từ last_valid = 10, dự đoán temp:0, abc, temp:70 sẽ để lại số nào.

Đã học cơ bản · Theo dõi trạng thái nhận

Không đồng nhất màn hình trống sau cuộn với biến bị xóa. Mỗi bản tin cần có quyết định nhận/loại và lý do, còn trường hợp None cần ghi là chưa có dữ liệu mới.

Lập bảng bản tin, lý do, giá trị trước và sau cho temp:-21, temp:-20, temp:60.

Muốn tìm hiểu sâu · Thiết kế hạn dùng dữ liệu

Đặt trước một thời hạn theo nhiệm vụ, ghi thời điểm nhận hợp lệ cuối và có trạng thái “cũ”. Việc nhận bản tin lỗi không nên làm mới tuổi của phép đo đúng cũ.

Đề xuất thời hạn 5 s cho ví dụ giả định: nêu đầu ra ở 2 s và 6 s sau bản tin hợp lệ cuối, rồi giải thích giới hạn bảo mật vẫn còn.

Kiến thức trọng tâm

Thống nhất một chuỗi trước khi truyền

Hai bên dùng MicroPython, bật radio bằng radio.on() và đặt radio.config(group=7). radio.send("temp:23") gửi chuỗi: phần trước dấu : là tên temp, phần sau là số theo đơn vị giả định °C. Bên nhận dùng radio.receive(); None nghĩa là chưa có bản tin để đọc, không phải nhiệt độ 0. Nhóm chọn bản tin để xử lý, không mã hóa hay xác thực người gửi.

Kiểm tra trước khi cập nhật

message.split(":", 1) tách tên và dữ liệu; float(raw) chuyển phần số sang giá trị. Chỉ tên temp với giá trị trong miền đóng [-20,60] mới cập nhật last_valid; sai định dạng hoặc ngoài miền thì giữ giá trị cũ. Ví dụ temp:-20 và temp:60 hợp lệ, temp:999 và other:22 không hợp lệ. Miền này là quy tắc của bài, không phải thông số độ chính xác của cảm biến. last_valid là trạng thái được lưu trong biến; số cuộn trên màn hình chỉ là cách trình bày tạm thời. Để kiểm tra trạng thái sau bản tin lỗi, cần đầu ra print thử nghiệm hoặc đối chiếu mã, không chỉ ảnh màn hình trống.

Đọc thêm · Dữ liệu cũ và các giao thức khác

Radio có thể mất gói, nhiễu và có giới hạn phạm vi; simulator không đo được tầm xa thật. Một sản phẩm cần ghi thời điểm nhận cuối và báo dữ liệu cũ khi quá hạn do thiết kế quy định. Tên trường và nhóm không bảo vệ dữ liệu nhạy cảm; không gửi thông tin cá nhân qua ví dụ này. Radio cơ bản dùng được trên micro:bit V1 và V2, nhưng tính năng V2 như microphone/speaker không được giả định có trên V1. Mẫu dùng chuỗi MicroPython, khác định dạng MakeCode radio.sendValue. Biến thể MakeCode chỉ dùng như một cặp chương trình riêng trong MakeCode; không ghép với bên gửi hoặc nhận MicroPython của bài.

Ví dụ có lời giải

Giá trị đang lưu là 21. Chuỗi temp:23 hợp lệ → 23; temp:999 sai miền → vẫn 23; other:22 sai tên → vẫn 23; temp:-20 hợp lệ ở biên → -20. Nếu chưa có bản tin mới, radio.receive() trả None; không được kết luận nhiệt độ bằng 0.

Lỗi thường gặp và cách tránh

1. Cùng nhóm radio không có nghĩa bản tin đã được mã hóa hoặc người gửi đã được xác thực.

2. Một số hợp lệ theo giao thức không tự chứng minh đó là số đo cảm biến thật hay còn mới.

Các bước thực hành

1. Dự đoán: viết quy ước nhóm 7, chuỗi dạng temp:23, đơn vị giả định °C và miền -20..60. Từ giá trị cũ 21, đoán kết quả của temp:23, temp:999, other:22 và temp:-20.

2. Thao tác: chạy mã bên nhận MicroPython trong Lab → Python Editor. Mở bảng Radio, chọn cùng nhóm 7 và gửi chuỗi temp:23 để kiểm tra bộ nhận.

3. Quan sát: gửi lần lượt temp:-20, temp:60, temp:-21, temp:999, other:22 và chuỗi sai định dạng. Bản tin hợp lệ làm xuất hiện lượt cuộn số mới. Mã gốc chỉ cuộn tạm thời, nên màn hình trống sau đó chưa chứng minh last_valid đã bị xóa.

4. Kiểm tra giá trị lưu: trong bản sao mã Python, thêm print(last_valid) ngay trước sleep(20), thụt vào 4 dấu cách trong while True. Đọc đầu ra print của editor khi gửi từng bản tin: bản tin bị loại phải giữ số cũ. Dừng sau khi ghi đủ dữ liệu rồi xóa dòng print thử nghiệm.

5. Nếu có hai bo: nạp mã bên nhận lên một bo và biến thể MicroPython bên gửi lên bo kia. Bấm A ở bên gửi để gửi temp:23; cả hai dùng nhóm 7 và cùng định dạng chuỗi. Hai tab editor độc lập không tự ghép thành một cặp radio.

6. Giải thích: ngừng gửi và nêu vì sao giá trị cũ chưa phải số đo mới hay bằng 0. Viết yêu cầu hết hạn dữ liệu cho phần nâng cao; mã hiện tại chưa thực hiện yêu cầu đó.

Mã mẫu · MicroPython · bên nhận radio

```
from microbit import *
import radio

# RECEIVER: both devices must run the same MicroPython text protocol.
# MakeCode sendValue packets use another format; do not mix the programs.
radio.config(group=7)
radio.on()
last_valid = None
while True:
  try:
    message = radio.receive()
    if message is not None:
      name, raw = message.split(":", 1)
      value = float(raw)
      if name == "temp" and -20 <= value <= 60:
        last_valid = value
        display.scroll(str(last_valid), wait=False, loop=False)
  except (ValueError, TypeError):
    # Ignore malformed/non-text data; keep the last valid value.
    pass
  sleep(20)
```

MicroPython · bên gửi radio · MicroPython · bên gửi radio

```
from microbit import *
import radio

radio.config(group=7)
radio.on()
while True:
  if button_a.was_pressed():
    radio.send("temp:23")
    display.show(Image.YES)
  if button_b.was_pressed():
    display.clear()
  sleep(20)
```

Thay thế · MakeCode JavaScript · MakeCode JavaScript · hai dự án gửi/nhận riêng

```
// CHUONG TRINH BEN NHAN: dan vao mot du an MakeCode rieng.
let lastValid = 0
radio.setGroup(7)
radio.onReceivedValue(function (name, value) {
  if (name == "temp" && value >= -20 && value <= 60) {
    lastValid = value
    basic.showNumber(lastValid)
  }
})

// BEN GUI: tao du an khac, khong ghep nguyen hai chuong trinh.
// radio.setGroup(7)
// input.onButtonPressed(Button.A, function () {
//     radio.sendValue("temp", 23)
// })
```

Chọn và sử dụng phòng lab

Trong Lab → micro:bit Python Editor, chạy mã bên nhận MicroPython nhóm 7. Dùng bảng Radio cùng nhóm gửi temp:23, temp:-20, temp:60 rồi temp:999 và other:22. Số chỉ cuộn tạm thời khi nhận hợp lệ; màn hình trống sau cuộn chưa cho biết giá trị lưu. Muốn kiểm tra last_valid sau bản tin bị loại, trong bản sao thêm print(last_valid) ngay trước sleep(20), cùng mức thụt vào 4 dấu cách, rồi đọc đầu ra print; dừng và xóa dòng này khi ghi đủ dữ liệu. Muốn thử hai bo thật, nạp biến thể MicroPython bên gửi lên bo thứ hai. Hai bên dùng cùng chuỗi MicroPython; MakeCode sendValue có định dạng khác. Bàn bản tin trong trang chỉ mô phỏng quy tắc lọc, không phát radio; hai tab editor không tự liên lạc.

Bài tập áp dụng

Từ giá trị cũ 21, dự đoán giá trị lưu sau temp:23, temp:999, other:22, temp:-20. Thử quy tắc trên bàn mô hình bản tin; khi thử mã MicroPython, thêm tạm print(last_valid) trước sleep(20) trong vòng lặp để quan sát trạng thái sau từng bản tin. Phân biệt giá trị lưu với hình cuộn tạm thời. Đề xuất chính sách dữ liệu quá hạn.

Gợi ý đối chiếu sau khi tự làm

Các giá trị lưu lần lượt 23,23,23,-20. temp:999 sai miền, other:22 sai tên nên không ghi đè. Có thể yêu cầu hiện “dữ liệu cũ” sau một thời hạn không nhận mẫu hợp lệ; mã hiện tại mới lọc định dạng, tên và miền, chưa theo dõi thời điểm nhận hay xác thực người gửi.

Tự kiểm tra mức hiểu

1. Vì sao temp:60 được nhận nhưng temp:60.1 bị loại theo quy tắc này?

2. Khi lâu không nhận bản tin, mã cần thêm trạng thái gì để báo dữ liệu cũ?

Câu hỏi trắc nghiệm

Đặt cùng nhóm radio có mã hóa bản tin không?

1. Có

2. Không

3. Chỉ trên máy tính

Đáp án: 2. Nhóm radio là cơ chế chọn/lọc nhóm truyền trong API này; nó không thực hiện mã hóa hoặc bảo đảm nguồn gửi đáng tin cậy.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

MicroPython micro:bit — API chính thức

https://microbit-micropython.readthedocs.io/en/v2-docs/

MicroPython micro:bit — module radio và định dạng chuỗi

https://microbit-micropython.readthedocs.io/en/v2-docs/radio.html

micro:bit Python Editor — soạn thảo và mô phỏng

https://python.microbit.org/v/3/

Thay thế · MakeCode micro:bit — nhóm radio

https://makecode.microbit.org/reference/radio/set-group

Thay thế · MakeCode — nhận cặp tên và số

https://makecode.microbit.org/reference/radio/on-received-value

Bài 6/6 · Dự án: bộ đếm có kiểm thử sự kiện

Hoàn thiện bộ đếm thành một dự án có yêu cầu, bộ ca kiểm thử và báo cáo lỗi có thể tái hiện.

Chuẩn bị

Đã chạy bộ đếm MicroPython, biết count, get_presses(), was_pressed() và quy tắc ưu tiên B. Chuẩn bị bảng trạng thái đầu, thao tác, kỳ vọng, quan sát.

Sau bài học, bạn có thể

1. Đặc tả trạng thái khởi tạo, thao tác tăng và đặt lại.

2. Lập và chạy bộ kiểm thử có kết quả kỳ vọng từng bước.

3. Phân tích lỗi bằng chuỗi thao tác tái hiện thay vì chỉ nói chương trình không chạy.

Cùng hiểu bài từ đầu

Một dự án hoàn chỉnh cần hơn một lần chạy thấy đúng. Bạn sẽ viết điều mong đợi trước, ghi kết quả thực sau từng thao tác, rồi dùng chênh lệch để tìm lỗi. Với bộ đếm, phần khó thường nằm ở lịch sử: một lần đặt lại có thể che lỗi cộng dồn phía trước. Vì vậy báo cáo phải có các mốc trung gian và ca khởi động lại, chứ không chỉ ảnh chụp số cuối.

Hãy coi bộ ca thử như công thức kiểm tra một chiếc cân: đặt vật nào, theo thứ tự nào, mong chờ gì đều được ghi trước.

Ca thử bộ đếm cũng cần trạng thái đầu, chuỗi nút và giá trị sau từng nút. Người khác làm lại cùng ca có thể xác nhận lỗi hoặc bản sửa.

Giới hạn của ví von: Một vài ca đúng không chứng minh mọi cách bấm đều đúng. Bấm gần nhau và ưu tiên cùng lượt đọc cần ca riêng; mẫu MicroPython không tự đại diện cho cơ chế sự kiện MakeCode.

Những từ cần hiểu

1. Yêu cầu: Mệnh đề có thể kiểm tra, như khởi động bằng 0, A cộng số lần đã nhận, B đặt lại và B ưu tiên nếu cùng lượt. Viết trước khi thử để có chuẩn so sánh.

2. Ca kiểm thử: Một bộ gồm trạng thái đầu, đầu vào có thứ tự, kết quả kỳ vọng và mốc quan sát. Ca càng rõ càng dễ lặp lại.

3. Lỗi có thể tái hiện: Sai khác xuất hiện khi người khác thực hiện được cùng các bước từ cùng trạng thái đầu. “Có lúc số sai” chưa đủ để tìm nguyên nhân.

4. Kiểm tra lại sau sửa: Chạy lại ca từng lỗi và những ca từng đúng. Một thay đổi có thể chữa tăng đếm nhưng làm hỏng reset.

Phát hiện lỗi bị kết quả cuối che khuất

Bắt đầu với: Yêu cầu A tăng một khi bấm rời nhau, B đặt lại. Bản bị lỗi đặt count = 0 ngay trước phép cộng trong nhánh A.

Viết kỳ vọng trước

Từ 0, ca A,A,B phải cho 1,2,0. Ghi cả ba số; số 2 là bằng chứng bộ đếm đã nhớ lần A trước.

Chạy bản lỗi

Quan sát được 1,1,0. Kết quả cuối 0 vẫn đúng, nhưng mốc thứ hai sai: bộ đếm đã xóa lịch sử trước khi cộng.

Rút ca tái hiện ngắn

A,A từ 0 đã đủ: mong 1,2 nhưng thấy 1,1. Ca ngắn này chỉ ra lỗi ở tích lũy, không cần kéo thêm B vào.

Sửa và kiểm tra lại

Bỏ dòng khởi tạo sai trong nhánh A, giữ khởi tạo trước vòng lặp. Chạy lại A,A và ca reset B,B,A; sau đó thử Run lại để xác nhận về 0.

Kết quả: Báo cáo có thể ghi: “Từ count = 0, bấm A hai lần rời nhau; kỳ vọng 1,2; quan sát 1,1; sau sửa quan sát 1,2.”

Quan sát trung gian giúp định vị mất trạng thái. Thử lại các ca khác bảo đảm sửa đúng nguyên nhân mà vẫn giữ các yêu cầu còn lại; đây là bằng chứng trong phạm vi bộ thử đã dùng.

Tự giải thích rồi đối chiếu

Vì sao không sửa cột kỳ vọng thành 1,1 sau khi thấy bản lỗi chạy như vậy?

Kỳ vọng xuất phát từ yêu cầu đã chọn. Đổi kỳ vọng theo lỗi sẽ mất chuẩn đánh giá; chỉ đổi yêu cầu khi có quyết định thiết kế rõ ràng.

Chạy lại chương trình từ count = 3 cần mong số nào?

Mẫu khởi tạo count = 0 nên về 0. Giữ số sau mất nguồn là một yêu cầu mới chưa được mẫu thực hiện.

Chọn độ sâu phù hợp

Mới bắt đầu · Hoàn thành bốn ca rõ ràng

Bắt đầu bằng bấm chậm: tăng, reset khi 0, reset sau tăng, khởi động lại. Mỗi hàng ghi kỳ vọng rồi mới điền quan sát.

Viết ca B,B từ 0 và ca A,A,Run lại, ghi đủ trạng thái sau từng thao tác.

Đã học cơ bản · Viết báo cáo lỗi hữu ích

Báo cáo cần bản mã đang dùng, trạng thái đầu, bước ngắn nhất, kỳ vọng và quan sát. Khi số đang cuộn, ghi mốc đợi đọc đủ số để người khác lặp lại được.

Tạo bản lỗi trong bản sao, báo cáo A,A, rồi xác nhận lại bằng bản đã khôi phục.

Muốn tìm hiểu sâu · Kiểm thử chính sách cùng lượt

Trong mẫu, presses được đọc rồi reset được kiểm tra; reset thắng khi cả hai cùng được nhận trong một lượt. Ca bấm chậm không kiểm tra được hết tình huống này.

Thiết kế bảng đầu vào (presses,reset) và kỳ vọng; tách kết quả chạy tay chắc chắn với kết quả bấm gần nhau chưa quan sát rõ.

Kiến thức trọng tâm

Yêu cầu phải khớp mã MicroPython

count khởi tạo 0 trước while True. Chương trình lấy presses = button_a.get_presses() và reset = button_b.was_pressed() mỗi lượt; nếu reset thì count = 0, nếu không thì cộng presses. Vì vậy A/B cùng được ghi nhận trong một lượt đọc sẽ cho reset, các lần A của lượt đó không được cộng. Chạy lại chương trình về 0; mẫu chưa có lưu sau mất nguồn.

Ca thử cần thứ tự và mốc quan sát

Mỗi ca ghi trạng thái ban đầu, chuỗi nút, kết quả kỳ vọng và quan sát sau từng thao tác. Thử tăng, reset lúc 0, reset sau khi tăng và chạy lại. Mẫu dùng display.scroll(..., wait=False) nên vòng đọc nút không chờ cuộn xong; bấm chậm ở bộ ca đầu để đọc đủ số. Thử bấm gần nhau ở ca riêng và đối chiếu chính sách B thắng trong cùng lượt đọc.

Đọc thêm · Tái hiện và kiểm tra lại sau khi sửa

Thêm count = 0 ngay trước count += presses trong nhánh A tạo lỗi: hai lần A rời nhau cho 1,1 thay vì 1,2. Ghi ca ngắn nhất, chỉ sửa một nguyên nhân rồi chạy lại cả ca lỗi lẫn ca từng đúng. Nhật ký ghi kết quả quan sát kể cả chưa đạt. Biến thể MakeCode có cơ chế sự kiện riêng; ca về ưu tiên cùng lượt đọc đang mô tả mẫu MicroPython, không tự áp dụng cho MakeCode.

Ví dụ có lời giải

Ca 1 từ 0: A,A,B,A → 1,2,0,1. Ca 2 từ 0: B,B,A,A,A → 0,0,1,2,3. Ca 3: đang 3 rồi khởi động lại → 0. Bảng có từng trạng thái trung gian sẽ phát hiện lỗi dù một ca tình cờ vẫn có kết quả cuối đúng.

Lỗi thường gặp và cách tránh

1. Một ca chạy đúng không đủ chứng minh mọi chuỗi sự kiện đều đúng.

2. Không tự tuyên bố hỗ trợ bấm nhanh/đồng thời hoặc lưu sau mất nguồn khi chưa đặc tả và kiểm thử.

Các bước thực hành

1. Dự đoán: viết yêu cầu khởi động = 0, A tăng theo số lần bấm đã đọc, B reset, chạy lại về 0; thêm quy tắc B thắng khi A/B cùng được ghi nhận trong một lượt đọc.

2. Thao tác: dùng bản dự án MicroPython của bài trong Lab → Python Editor, chạy mã gốc trước khi chỉnh và kiểm tra ban đầu là 0.

3. Quan sát: thực hiện A,A,B,A rồi B,B,A,A,A từng bước, đợi đọc số trước lần tiếp theo. Ghi kỳ vọng và quan sát ở từng mốc, sau đó Run lại để kiểm tra về 0.

4. Trong bản sao, thêm count = 0 ngay trước count += presses ở nhánh A để tạo lỗi; thử A,A, ghi chuỗi tái hiện ngắn nhất rồi xóa dòng thêm.

5. Chạy lại các ca đã ghi. Thử thêm thao tác A/B gần nhau và đối chiếu quy tắc ưu tiên reset; ghi trường hợp chưa quan sát chắc thay vì tự kết luận đạt.

6. Giải thích: viết báo cáo ngắn yêu cầu đạt/chưa đạt, cách tái hiện lỗi và một mở rộng cần đặc tả riêng, chẳng hạn lưu số sau mất nguồn.

Mã mẫu · MicroPython · micro:bit Python Editor

```
from microbit import *

# Specification: boot=0, A adds one, B resets, reset is repeatable.
# Sequential tests: wait for each display before the next input.
# Fast/simultaneous A+B policy: reset wins for that polling interval.
count = 0
display.show(str(count))
while True:
  presses = button_a.get_presses()
  reset = button_b.was_pressed()
  if reset:
    count = 0
  elif presses:
    count += presses
  if reset or presses:
    display.scroll(str(count), wait=False, loop=False)
  sleep(20)
```

Thay thế · MakeCode JavaScript · MakeCode JavaScript · phiên bản bộ đếm cơ bản để kiểm thử

```
let count = 0
basic.showNumber(count)
input.onButtonPressed(Button.A, function () {
  count += 1
  basic.showNumber(count)
})
input.onButtonPressed(Button.B, function () {
  count = 0
  basic.showNumber(count)
})
```

Chọn và sử dụng phòng lab

Trong Lab → micro:bit Python Editor, Run bộ đếm và thử khởi động = 0; A,A = 2; B = 0; B,B vẫn 0; Run lại = 0. Mỗi ca ghi từng trạng thái, bấm chậm để đọc rõ số. Sau đó thử A/B gần nhau: get_presses() đã lấy số lần A trong lượt đọc, nhưng nhánh B được ưu tiên và bỏ qua phần cộng của lượt đó. Mã cuộn số không chặn nên hình có thể đổi trước khi cuộn xong khi bấm nhanh. Lưu bản sao trước khi chủ đích tạo lỗi; bàn kiểm thử trong trang chỉ đối chiếu quy tắc từ chuỗi thao tác, không thực thi Python.

Bài tập áp dụng

Viết ít nhất 4 ca có trạng thái đầu, chuỗi thao tác, từng trạng thái kỳ vọng và quan sát: tăng, reset khi 0, reset sau khi tăng, khởi động lại. Thêm báo cáo một lỗi với cách tái hiện.

Gợi ý đối chiếu sau khi tự làm

Ví dụ: A,A → 1,2; B,B từ 0 → 0,0; A,A,B → 1,2,0; chạy lại từ 2 → 0. Nếu thêm count = 0 ngay trước count += presses trong nhánh A, hai lần A rời nhau cho 1,1. Xóa dòng thêm để chỉ khởi tạo một lần trước while True rồi chạy lại các ca. Nếu A/B cùng được ghi nhận trong một lượt đọc, kết quả của lượt đó là 0 theo quy tắc ưu tiên B.

Tự kiểm tra mức hiểu

1. Nếu chỉ ghi kết quả cuối, một lỗi trung gian có thể bị bỏ sót thế nào?

2. Sau khi sửa một lỗi, vì sao phải chạy lại những ca trước đây đã đúng?

Câu hỏi trắc nghiệm

A,A,B,A cho kết quả...

1. 0

2. 1

3. 3

Đáp án: 2. Thứ tự A,A,B,A cho 1,2,0,1: sự kiện B đặt lại trạng thái, vì thế lần A cuối bắt đầu từ 0.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

MicroPython micro:bit — API chính thức

https://microbit-micropython.readthedocs.io/en/v2-docs/

MicroPython micro:bit — nút bấm và get_presses

https://microbit-micropython.readthedocs.io/en/v2-docs/button.html

micro:bit Python Editor — soạn thảo và mô phỏng

https://python.microbit.org/v/3/

Thay thế · MakeCode micro:bit — sự kiện nút bấm

https://makecode.microbit.org/reference/input/on-button-pressed

Thay thế · MakeCode — hiển thị số và thời gian hiển thị

https://makecode.microbit.org/reference/basic/show-number